

Troubleshooting Guide

This guide helps resolve common issues encountered when using the Azure DevOps Permission Matrix Toolkit.

Common Issues and Solutions

Authentication and Access Issues

Issue: “Unauthorized” or “Access Denied” Errors

Symptoms:

```
Error: The remote server returned an error: (401) Unauthorized
Error: Access denied. Check your permissions and try again
```

Possible Causes and Solutions:

1. Expired Personal Access Token

```
powershell
# Check token expiration in Azure DevOps
# Navigate to: User Settings > Personal Access Tokens
# Verify token is not expired and regenerate if needed
```

2. Insufficient Token Scopes

- Verify your PAT includes all required scopes:

- Project and Team (Read)
- Identity (Read)
- Security (Manage)
- Code (Read)
- Build (Read)
- Release (Read)
- Service Connections (Read)
- Agent Pools (Read)

3. Incorrect Organization URL

```
``powershell
# Correct format
$orgUrl = "https://dev.azure.com/yourorganization"

# Common mistakes to avoid
$wrongUrl1 = "https://yourorganization.visualstudio.com" # Old format
$wrongUrl2 = "https://dev.azure.com/yourorganization/" # Trailing slash
```
```

#### 1. User Not Member of Organization

- Verify the user account has access to the Azure DevOps organization
- Check if user is restricted by conditional access policies

## Issue: “Forbidden” Errors (403)

### Symptoms:

```
Error: The remote server returned an error: (403) Forbidden
```

### Solutions:

#### 1. Check User Permissions

```
powershell
Verify user has at least "Basic" access level
Check if user is member of required security groups
```

#### 2. Organization Policies

- Check if organization has restricted API access
- Verify third-party application access policies

## PowerShell Execution Issues

### Issue: “Execution of scripts is disabled”

### Symptoms:

```
.\Get-AzureDevOpsOrgPermissions.ps1 : File cannot be loaded because running scripts is
disabled on this system
```

### Solutions:

#### 1. Set Execution Policy (Recommended)

```
```powershell
# For current user only (safer)
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# For all users (requires admin)
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope LocalMachine
```
```

#### 1. Bypass for Single Execution

```
powershell
powershell.exe -ExecutionPolicy Bypass -File ".\Get-AzureDevOpsOrgPermissions.ps1"
```

#### 2. Unblock Downloaded Files

```
powershell
Unblock all script files
Get-ChildItem -Path ".\scripts*.ps1" | Unblock-File
```

### Issue: “ImportExcel module not found”

### Symptoms:

```
Import-Module : The specified module 'ImportExcel' was not loaded
```

**Solutions:****1. Install Module**

```

```powershell
# Install for current user
Install-Module -Name ImportExcel -Scope CurrentUser -Force

# If behind corporate firewall
Install-Module -Name ImportExcel -Scope CurrentUser -Force -Repository PSGallery -Proxy "http://
proxy.company.com:8080"
```

```

**1. Manual Installation**

```

powershell
Download and install manually
Save-Module -Name ImportExcel -Path "C:\Temp\Modules"
Copy-Item "C:\Temp\Modules\ImportExcel" -Destination
"$env:USERPROFILE\Documents\PowerShell\Modules\" -Recurse

```

**2. Skip Excel Generation**

```

powershell
Run without Excel report generation
.\Run-AzureDevOpsPermissionAudit.ps1 -OrganizationUrl $orgUrl -PersonalAccessToken $pat
(Remove -GenerateExcelReport parameter)

```

**Network and Connectivity Issues****Issue: “Unable to connect to the remote server”****Symptoms:**

```

Error: Unable to connect to the remote server
Error: The operation has timed out

```

**Solutions:****1. Check Internet Connectivity**

```

```powershell
# Test basic connectivity
Test-NetConnection -ComputerName "dev.azure.com" -Port 443

# Test with PowerShell
Invoke-WebRequest -Uri "https://dev.azure.com" -UseBasicParsing
```

```

**1. Configure Proxy Settings**

```

```powershell
# Set proxy for current session
$proxy = New-Object System.Net.WebProxy("http://proxy.company.com:8080")
$proxy.Credentials = [System.Net.CredentialCache]::DefaultCredentials
[System.Net.WebRequest]::DefaultWebProxy = $proxy

```

```
# Or set system proxy
netsh winhttp set proxy proxy.company.com:8080
...
```

1. Increase Timeout Values

```
powershell
# Add to script parameters or modify scripts
$PSDefaultParameterValues['Invoke-RestMethod:TimeoutSec'] = 300
```

2. TLS/SSL Issues

```
powershell
# Force TLS 1.2
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
```

Data and Output Issues

Issue: “Empty or incomplete CSV files”

Symptoms:

- CSV files are created but contain no data or headers only
- Missing expected permissions or groups

Solutions:

1. Check API Permissions

```
powershell
# Verify token has correct scopes
# Test with minimal API call first
$headers = @{Authorization = "Basic $($Convert::ToBase64String([Text.Encoding]::ASCII.GetBytes(":$pat")))}
Invoke-RestMethod -Uri "$orgUrl/_apis/projects?api-version=7.1-preview.4" -Headers $headers
```

2. Verify Organization/Project Names

```
powershell
# List available projects
$projects = Invoke-RestMethod -Uri "$orgUrl/_apis/projects?api-version=7.1-preview.4" -Headers $headers
$projects.value | Select-Object name, id
```

3. Check for API Rate Limiting

```
powershell
# Add delays between API calls if needed
Start-Sleep -Seconds 1 # Add to scripts between API calls
```

Issue: “Excel file generation fails”

Symptoms:

```
Error: Export-Excel : Cannot bind argument to parameter 'Path'
Error: The process cannot access the file because it is being used by another process
```

Solutions:**1. Close Excel Applications**

```
powershell
# Close any open Excel files
Get-Process excel -ErrorAction SilentlyContinue | Stop-Process -Force
```

2. Check File Permissions

```
```powershell
Verify write permissions to output directory
Test-Path -Path $OutputDirectory -PathType Container

```
```

3. Use Alternative Output Path

```
powershell
# Try different output location
$OutputDirectory = "$env:TEMP\AzureDevOpsAudit"
```

Performance Issues**Issue: “Scripts running very slowly”****Symptoms:**

- Scripts take excessive time to complete
- Timeouts on large organizations

Solutions:**1. Optimize for Large Organizations**

```
powershell
# Analyze specific projects instead of all
.\Run-AzureDevOpsPermissionAudit.ps1 -ProjectName "SpecificProject" -OrganizationUrl
$orgUrl -PersonalAccessToken $pat
```

2. Increase PowerShell Memory

```
```powershell
For PowerShell 5.1
powershell.exe -Command "& {Set-Variable -Name 'MaximumHistoryCount' -Value 1}"

For PowerShell 7+
pwsh -Command "& {System.GC (:Collect())::Collect()}"

```
```

1. Run Scripts in Background

```
powershell
# Use background jobs for long-running operations
Start-Job -ScriptBlock {
    & "C:\Tools\AzureDevOpsToolkit\scripts\Run-AzureDevOpsPermissionAudit.ps1" -
    OrganizationUrl $using:orgUrl -PersonalAccessToken $using:pat
}
```

API and Data Issues**Issue: “API version not supported”****Symptoms:**

```
Error: API version '7.1-preview.1' is not supported
```

Solutions:

1. Update API Versions

```
powershell
# Check current API versions
# Modify scripts to use stable API versions if needed
# Replace "7.1-preview.1" with "6.0" for stable versions
```

2. Check Azure DevOps Service Updates

- Verify Azure DevOps Services status
- Check for service updates that might affect API versions

Issue: “Inconsistent permission data”

Symptoms:

- Different results between script runs
- Missing permissions that should exist

Solutions:

1. Check Caching Issues

```
powershell
# Clear PowerShell session and restart
Remove-Variable * -ErrorAction SilentlyContinue
```

2. Verify Timing Issues

```
powershell
# Add delays between API calls
Start-Sleep -Milliseconds 500
```

3. Check for Concurrent Modifications

- Ensure no other processes are modifying permissions during audit
- Run audit during maintenance windows

Debugging Techniques

Enable Verbose Logging

```
# Add to scripts for detailed logging
$VerbosePreference = "Continue"
$DebugPreference = "Continue"

# Or run with verbose flag
.\Get-AzureDevOpsOrgPermissions.ps1 -Verbose -Debug
```

Test Individual Components

```
# Test organization connectivity
$headers = @{Authorization = "Basic $($([Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes(":$pat"))))"}
$response = Invoke-RestMethod -Uri "$orgUrl/_apis/connectionData" -Headers $headers
$response.authenticatedUser

# Test specific API endpoints
$projects = Invoke-RestMethod -Uri "$orgUrl/_apis/projects?api-version=7.1-preview.4" -
Headers $headers
$projects.count
```

Capture Network Traffic

```
# Enable web request tracing (PowerShell 6+)
$PSDefaultParameterValues['Invoke-RestMethod:Verbose'] = $true

# Or use Fiddler/Wireshark for detailed network analysis
```

Getting Additional Help

Log Collection

When reporting issues, collect the following information:

1. PowerShell Version

```
powershell
$PSVersionTable
```

2. Module Versions

```
powershell
Get-Module -ListAvailable | Where-Object {$_.Name -like "*Excel*" -or $_.Name -like
"*Azure*"}
```

3. Error Details

```
powershell
# Capture full error information
$error[0] | Format-List * -Force
```

4. Environment Information

```
powershell
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion, TotalPhysicalMemory
```

Common Workarounds

1. For Large Organizations

- Process projects individually
- Use filtering parameters where available
- Schedule during off-peak hours

2. For Network Issues

- Use VPN if accessing from external networks

- Configure proxy settings appropriately
- Verify firewall rules allow HTTPS traffic

3. For Permission Issues

- Use organization administrator account for initial setup
- Verify user is not restricted by conditional access
- Check Azure AD group memberships

Contact Information

For additional support:

- Review Azure DevOps Services documentation
- Check Azure DevOps Developer Community forums
- Verify service status at <https://status.dev.azure.com/>

If issues persist after trying these solutions, collect the diagnostic information above and contact your system administrator or Azure DevOps support.